Payment Gateway The complete request flow

Implementation walkthrough | 10 November 2026
Includes the shared PaymentResponse record, bank-owned expiry serialization and current logging levels. This describes the source code; running processes must be restarted or rebuilt after changes.

1. Start the application

PaymentGatewayApplication starts Spring Boot. Component scanning wires the controller, payment service, repository, idempotency coordinator, bank client and observability components. Constructor injection provides their dependencies. The repository and idempotency map begin empty.

BankClientConfiguration creates a RestTemplate with 10-second connect and read timeouts. Its interceptor forwards the current correlation ID to the bank. The local base URL is http://localhost:8080; Compose overrides it to http://bank_simulator:8080.

The gateway listens internally on 8090. IntelliJ runs it locally on 8090; Docker can publish it on host port 8091 so both can coexist. Port 2525 belongs to the simulator administration API, not payment processing.

2. Understand the boundaries

Area	Responsibility
payment	Merchant API, validation, idempotency, request mapping and masked result storage.
bank	External request/response DTOs, transport and strict response parsing. No dependency on payment classes.
observability	Correlation filter and shared metric names. Logs and metrics cross the request flow.

Successful POST: the call sequence

```
Merchant -> CorrelationIdFilter -> Controller (@Valid)
  -> PaymentGatewayService -> PaymentIdempotencyService
  -> processNewPayment -> BankPaymentRequestMapper
  -> BankClient -> Docker simulator -> BankResponseMapper
  -> save PaymentResponse -> complete idempotency outcome
  -> controller lookup -> HTTP 201 -> completion log
```

The service supplies the processing function to the idempotency coordinator. Only a caller selected to process a new attempt executes that function. GET takes the shorter controller -> service -> repository path.

Receive and validate a payment

3. Establish correlation

CorrelationIdFilter applies to /payment and its subpaths. A canonical UUID in X-Correlation-ID is preserved; a missing or invalid value is replaced with a generated UUID. The filter returns this header and puts it in the logging MDC (thread-local log context). It starts timing before invoking the request chain.

4. Bind the merchant JSON

```
POST /payment
Content-Type: application/json
Idempotency-Key: demo-001

{"card_number":"2222405343240001",
 "expiry_month":12,"expiry_year":2099,
 "currency":"GBP","amount":1050,"cvv":"012"}
```

Jackson creates PostPaymentRequest. Card number and CVV are strings to preserve leading zeroes. Numeric fields use Integer so missing values remain null. Fractional-to-integer coercion is disabled. The merchant DTO no longer exposes a derived expiry_date property.

5. Validate at the HTTP boundary

Field	Implemented rule
Card number	Required; 14-19 ASCII digits. No Luhn check.
Expiry	Required; month 1-12, year 1-9999; combined month/year strictly later than the current month.
Currency	Required; exactly GBP, USD or EUR, uppercase.
Amount	Required Java Integer, in minor units. 1050 GBP means GBP 10.50. No positive-only restriction is imposed.
CVV	Required; 3-4 ASCII digits.

@Valid on the controller triggers these checks, including FutureExpiryValidator using the validation provider clock. Constructing a record or calling the service directly does not trigger validation. Response validation annotations are also not automatically executed.

Invalid input stops here

Malformed JSON or invalid fields go to GlobalExceptionHandler: HTTP 400 with status Rejected and the fixed message Invalid payment request. The bank is not called, no payment is saved and no idempotency key is reserved. The handler increments INVALID_REQUEST and logs a safe INFO message.

Coordinate idempotency and concurrency

6. Choose a new attempt or an existing result

The controller passes the validated request and optional Idempotency-Key to PaymentGatewayService.processPayment. The service calls PaymentIdempotencyService.execute with a supplier for the actual bank-and-save operation.

Situation	Behavior
No key	Record NO_KEY and execute a new attempt. Identical payloads without keys remain independent payments.
New key	Compute fingerprint; atomically reserve with putIfAbsent; record NEW. Only the successful reserving caller executes the supplier.
Same key, same fields	Record REPLAY and join the existing CompletableFuture. If work is in progress, wait; otherwise return its stored outcome immediately.
Same key, changed fields	Record CONFLICT and throw IdempotencyConflictException. HTTP 409; no additional bank call.

What the fingerprint contains

PaymentRequestFingerprint uses HMAC-SHA256 with a random 256-bit instance-local key. It includes full card number, expiry month/year, currency, amount and CVV. Fields are length-prefixed before hashing so field boundaries are unambiguous. A fresh mutable Mac is created for each call.

The cache retains the caller key, fingerprint and shared outcome future, not the raw payment request. A successful outcome holds the payment UUID. A known bank failure holds only a safe enum code, not upstream exceptions or bodies.

Why no extra locks are needed

ConcurrentHashMap.putIfAbsent is atomic. Same-key callers converge on one future; different keys can reach the bank concurrently. The repository also uses ConcurrentHashMap and stores immutable records. There is no ReentrantLock or ReadWriteLock spanning the external call.

Failure and lifetime semantics

Bank failures complete the shared outcome before being rethrown. A replay recreates the same safe failure without another attempt. Unexpected runtime failures also unblock waiters and keep the key reserved; later callers get an internal failure rather than a retry. This does not guarantee recovery from process termination.

All coordination is local to one process, with no TTL, eviction, merchant scope or distributed persistence. Restart loses it. A blank key currently behaves as a supplied key. Synchronous bank calls and waiting replays occupy request threads.

Call the bank and save the result

7. Build the bank-specific request

processNewPayment starts the payment-attempt timer. BankPaymentRequestMapper converts the validated merchant record to BankPaymentRequest and formats expiry with Locale.ROOT as MM/YYYY. Formatting now belongs entirely to this mapping boundary.

```
{"card_number":"2222405343240001",
 "expiry_date":"12/2099","currency":"GBP",
 "amount":1050,"cvv":"012"}
```

BankClient starts its own timer and POSTs to /payments using RestTemplate. This is a single synchronous call without automatic retries. Full card/CVV values must be sent to the simulator, but are not retained in the gateway repository or idempotency cache.

8. Interpret the simulator result

Odd final card digits produce an authorized bank response. Digits 2, 4, 6 and 8 produce a decline. Digit 0 produces HTTP 503. BankClient expects HTTP 200 for a normal outcome; BankResponseMapper then requires a JSON object containing an actual Boolean authorized value.

Strings such as "true", numbers, missing/null authorized fields and malformed bodies are rejected. authorization_code may be missing/null or a string; other types are rejected. The gateway does not expose or use this code as the payment ID.

9. Store an immutable masked response

The service maps true to AUTHORIZED and false to DECLINED, generates a UUID and extracts the final four card digits. It saves PaymentResponse in PaymentsRepository, keyed by that UUID. Both outcomes are stored. A bank exception exits before the repository write.

```
{"id":"<generated UUID>","status":"Authorized",
 "cardNumberLastFour":"0001",
 "expiryMonth":12,"expiryYear":2099,
 "currency":"GBP","amount":1050}
```

10. Return to the caller

The service logs the saved ID/outcome and returns the UUID. With a key, the coordinator completes the shared future with that ID. The controller calls getPaymentById and returns ResponseEntity<PaymentResponse> with HTTP 201. A decline also returns 201, with status Declined.

PaymentResponse is shared by POST and GET. It preserves the existing JSON fields, validation annotations and explicit toString. Historical expiry dates are valid in stored responses. No full card number or CVV is part of the response.

Retrieval, failures and request completion

11. Retrieve a previous payment

GET /payment/{id} passes through the same correlation filter. Spring converts the path ID to a UUID. The controller calls getPaymentById; the service reads the repository and returns the same PaymentResponse with HTTP 200. No bank call or idempotency decision is needed.

An absent UUID throws EventProcessingException and becomes HTTP 404 with {"message":"Page not found"}. A malformed UUID is rejected by framework parameter binding, rather than treated as a repository miss.

12. Translate failures consistently

Condition	Gateway result	Saved?
Invalid merchant input	400; status Rejected	No
Key used with different fields	409; IDEMPOTENCY_CONFLICT	No new result
Bank 503 or non-timeout connection failure	503; BANK_UNAVAILABLE	No
Connect/read timeout	504; BANK_TIMEOUT	No
Unexpected bank status or invalid response	502; BANK_INVALID_RESPONSE	No
Missing stored payment	404; Page not found	No change

Known bank errors use a code/message response with no payment ID or payment status. GlobalExceptionHandler uses @ResponseStatus mappings, records the safe error code and emits WARN. An upstream timeout is an unknown outcome, not proof that no payment happened.

A bank success followed by a process crash before storage cannot be reconciled by the in-memory implementation. Unexpected internal failures do not have a custom public error contract. Failure responses and successful PaymentResponse records deliberately have different shapes.

13. Complete the HTTP observation

The correlation filter logs method, route template, status, duration_ms and completed. Known responses, including handled bank failures, get INFO completion logs. Unexpected 5xx or escaping exceptions get ERROR. With completed=false, 500 is inferred rather than a confirmed response sent on the wire.

Finally, the filter restores the previous MDC value or removes the request ID so reused servlet threads do not retain another caller's context. Exceptions are not dumped by this filter; framework logs remain a separate redaction boundary.

Observe, demonstrate and explain the limits

14. Read the operational signals

Signal	Meaning
INFO	Payment saved; expected input/lookup/conflict messages; normal request completion.
WARN / ERROR	WARN for known bank technical failures. ERROR for unexpected HTTP failure completion.
DEBUG	Lookup ID, bank outcome/duration and idempotency decision. No key, fingerprint, payload or exception cause.
Timers	gateway.bank.requests and gateway.payment.attempts measure actual work, not replays.
Counters / gauge	gateway.idempotency.requests tracks decisions; gateway.payment.errors tracks handled errors; gateway.idempotency.entries tracks retained keys.

Actuator provides health, liveness, readiness, info, metrics and Prometheus export. HTTP/JVM metrics supplement application metrics. IDs are not metric labels. Readiness does not call the bank, and no Prometheus server or tracing collector is bundled.

15. Run the flow yourself

```
# Rebuild current source; use 8091 alongside IntelliJ on 8090.
GATEWAY_PORT=8091 docker compose up -d --build --wait
./scripts/smoke-test.py --base-url http://localhost:8091
docker compose logs -f --tail 50 gateway

# Individual paths:
./scripts/smoke/authorized.py --base-url http://localhost:8091
./scripts/smoke/replay.py --base-url http://localhost:8091
./scripts/smoke/bank_unavailable.py --base-url http://localhost:8091
```

The default smoke target is 8090: explicitly select 8091 when checking Docker logs. Locally, start only the simulator and choose Payment Gateway Local in IntelliJ with Gradle JVM 17. Avoid two local applications competing for 8090.

16. What is verified, and what remains limited

At the latest source verification, 120 Java tests passed. They cover validation, request/response contracts, bank failures, masking, concurrency, idempotency and logging. The 13 live smoke scripts cover the supplied simulator; timeout/malformed-bank scenarios use substituted transport in Java tests.

Repository and key state disappear on restart and can grow without bound. There is no authentication, TLS termination, merchant isolation, durable coordination or reconciliation. This is a local assessment implementation, not a distributed exactly-once payment system.

Code-reading route: PaymentGatewayController -> PaymentGatewayService -> PaymentIdempotencyService -> BankPaymentRequestMapper -> BankClient -> BankResponseMapper -> PaymentsRepository. Cross-cutting: CorrelationIdFilter, GlobalExceptionHandler and MetricNames. All are under src/main/java/com/checkout/payment/gateway/. Supporting guides: docs/architecture.md, docs/observability.md, docs/testing.md and docs/demo.md.